

```
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

public class AppLogger {

    // Path to the log file where all server activity is stored.
    private static final String LOG_FILE = "logs/server.log";

    // Shared timestamp format used for every log entry.
    private static final DateTimeFormatter FORMATTER =
        DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");

    // Central logging method used by all other helper methods.
    // It creates a timestamped log entry, prints it to the
console,
    // and appends it to the log file.
    private static void log(String level, String message) {
        String timestamp = LocalDateTime.now().format(FORMATTER);
        String logEntry = "[" + timestamp + "] [" + level + "] " +
message;

        // Print the log entry to the console for immediate
visibility.
        System.out.println(logEntry);

        // Append the same log entry to the log file.
        try (PrintWriter writer = new PrintWriter(new
BufferedWriter(new FileWriter(LOG_FILE, true)))) {
            writer.println(logEntry);
        } catch (IOException e) {
            // If file logging fails, print the error to standard
error.
            System.err.println("Failed to write to log file: " +
e.getMessage());
        }
    }
}
```

```
// Logs a general informational message.
public static void info(String message) {
    log("INFO", message);
}

// Logs a warning message for non-critical issues.
public static void warning(String message) {
    log("WARNING", message);
}

// Logs an error message for failures or unexpected problems.
public static void error(String message) {
    log("ERROR", message);
}

// Logs an error message and includes exception details.
public static void error(String message, Exception e) {
    log("ERROR", message + " | Exception: " +
e.getClass().getSimpleName() + " - " + e.getMessage());
}

// Logs when the server starts and records the port number.
public static void logServerStart(int port) {
    info("Server started on port " + port);
}

// Logs when the server shuts down.
public static void logServerStop() {
    info("Server stopped");
}

// Logs when a client successfully connects to the server.
public static void logClientConnected(String clientAddress) {
    info("Client connected: " + clientAddress);
}

// Logs the HTTP method and path for an incoming request.
public static void logRequest(String method, String path) {
    info("Request received: method=" + method + ", path=" +
path);
}
```

```

// Logs the path of a file that was successfully served.
public static void logFileServed(String filePath) {
    info("File served: " + filePath);
}

// Logs a 404 event when a requested path cannot be found.
public static void logNotFound(String path) {
    warning("404 Not Found: " + path);
}

// Logs details about a malformed or invalid request.
public static void logMalformedRequest(String details) {
    warning("Malformed request: " + details);
}

// Logs when a path is rejected for being unsafe.
public static void logUnsafePathRejected(String path) {
    warning("Rejected unsafe path: " + path);
}

// Logs why form validation failed.
public static void logFormValidationFailure(String reason) {
    warning("Form validation failure: " + reason);
}

// Logs when a form submission is successfully stored.
public static void logSubmissionStored(String details) {
    info("Submission stored: " + details);
}

// Logs internal server errors together with exception
details.
public static void logInternalError(String details, Exception
e) {
    error("Internal error: " + details, e);
}
}

```

```

import java.io.InputStreamReader;
import java.io.OutputStreamWriter;

```

```
import java.net.Socket;
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.IOException;

public class ConnectionHandler implements Runnable {
    // Each handler instance is responsible for one client socket
    connection.
    private final Socket socket;

    // Store the connected client socket so the request can be
    read and responded to.
    public ConnectionHandler(Socket socket) {
        this.socket = socket;
    }

    // Called when this handler is executed by a thread.
    // It delegates the actual work to the handle method.
    @Override
    public void run() {
        handle();
    }

    // Handles the full lifecycle of a single client request:
    // logging the connection, reading the HTTP request,
    // routing it, sending the response, and closing the socket.
    public void handle() {
        AppLogger.logClientConnected(socket.toString());

        try (
            // Reader for incoming text data from the client
            socket.
            BufferedReader reader = new BufferedReader(
                new InputStreamReader(socket.getInputStream())
            );
            // Writer for sending the HTTP response back to the
            client.
            BufferedWriter writer = new BufferedWriter(
                new OutputStreamWriter(socket.getOutputStream())
            )
        ) {
```

```

        HttpResponse response;

        try {
            // Read the first line of the HTTP request, such
as:

            // GET /index.html HTTP/1.1
            String requestLine = reader.readLine();
            System.out.println("REQUEST LINE = [" +
requestLine + "]");
            if (requestLine == null ||
requestLine.trim().isEmpty()) {
                throw new IllegalArgumentException("Request
line was empty");
            }

            // Build the full raw request so it can be passed
to the parser later.
            StringBuilder rawRequest = new StringBuilder();
            rawRequest.append(requestLine).append("\r\n");

            String line;
            int contentLength = 0;
            boolean blankLineFound = false;

            // Read request headers line by line until the
blank line is found.
            while ((line = reader.readLine()) != null) {
                rawRequest.append(line).append("\r\n");

                // Look for the Content-Length header so the
body size can be read correctly.
                if
(line.toLowerCase().startsWith("content-length:")) {
                    String value =
line.substring(line.indexOf(":") + 1).trim();

                    try {
                        contentLength =
Integer.parseInt(value);
                    } catch (NumberFormatException e) {

```

```
        throw new
IllegalArgumentException("Invalid Content-Length header");
    }

    // Negative body lengths are invalid and
should be rejected.
    if (contentLength < 0) {
        throw new
IllegalArgumentException("Content-Length cannot be negative");
    }
}

// A blank line marks the end of the headers
section.
    if (line.isEmpty()) {
        blankLineFound = true;
        break;
    }
}

// If no blank line was found, the request format
is incomplete.
    if (!blankLineFound) {
        throw new IllegalArgumentException("Missing
blank line after headers");
    }

    // If the request includes a body, read exactly
the number of characters declared.
    if (contentLength > 0) {
        char[] bodyChars = new char[contentLength];
        int totalRead = 0;

        while (totalRead < contentLength) {
            int charsRead = reader.read(bodyChars,
totalRead, contentLength - totalRead);

            if (charsRead == -1) {
                break;
            }
        }
    }
}
```

```
        totalRead += charsRead;
    }

    // Reject the request if the body received is
shorter than expected.
    if (totalRead < contentLength) {
        throw new
IllegalArgumentException("Request body was shorter than
Content-Length");
    }

    // Append the body to the raw request so the
parser receives the full message.
    rawRequest.append(bodyChars, 0, totalRead);
}

    // Parse the raw HTTP request into a structured
HttpRequest object.
    RequestParser parser = new
RequestParser(rawRequest.toString());
    HttpRequest request = parser.parse();

    // Pass the parsed request to the router to decide
how it should be handled.
    Router router = new Router();
    response = router.route(request);

    // As a safety fallback, generate a 500 response
if the router returns nothing.
    if (response == null) {
        response = new HttpResponse(
            "HTTP/1.1",
            500,
            "Internal Server Error",
            "Router returned no response"
        );
        response.addHeader("Content-Type",
"text/plain; charset=UTF-8");
    }

} catch (IllegalArgumentException e) {
```

```

        // Handle malformed client requests with a 400 Bad
Request response.
        AppLogger.logMalformedRequest(e.getMessage());

        response = new HttpResponseMessage(
            "HTTP/1.1",
            400,
            "Bad Request",
            e.getMessage()
        );
        response.addHeader("Content-Type", "text/plain;
charset=UTF-8");

    } catch (Exception e) {
        // Catch unexpected failures and return a
controlled 500 response.
        AppLogger.logInternalError("Unexpected failure in
ConnectionHandler", e);
        e.printStackTrace();

        response = new HttpResponseMessage(
            "HTTP/1.1",
            500,
            "Internal Server Error",
            "An internal server error occurred"
        );
        response.addHeader("Content-Type", "text/plain;
charset=UTF-8");
    }

    // Send the completed HTTP response to the client.
    writer.write(response.buildResponse());
    writer.flush();

} catch (IOException e) {
    // Log any low-level I/O errors that happen while
reading or writing.
    AppLogger.logInternalError("I/O failure in
ConnectionHandler", e);
    e.printStackTrace();
} finally {

```

```

        try {
            // Always close the socket after the request is
handled.
            socket.close();
        } catch (IOException e) {
            // Log any failure that occurs while closing the
socket.
            AppLogger.logInternalError("Failed to close
socket", e);
            e.printStackTrace();
        }
    }
}
}
}

```

```

import java.io.UnsupportedEncodingException;
import java.net.URLDecoder;
import java.util.HashMap;
import java.util.Map;

public class FormService {

    // Handles storing validated form submissions.
    private final SubmissionStore submissionStore;

    // Create a FormService and initialise the submission storage
dependency.
    public FormService() {
        this.submissionStore = new SubmissionStore();
    }

    // Handles form submissions sent in the request body,
typically from POST requests.
    public HttpResponse handlePost(HttpServletRequest request) {
        if (request == null) {
            return createResponse(400, "Bad Request", "Request was
null");
        }

        String body = request.getBody();
    }
}

```

```

        // Reject the request if no form body was provided.
        if (body == null || body.trim().isEmpty()) {
            return createResponse(400, "Bad Request", "Form body
is empty");
        }

        // Parse the raw form body into key-value pairs and pass
it for validation.
        Map<String, String> formData = parseFormData(body);
        return handleFormData(formData);
    }

    // Handles form submissions sent through query parameters,
typically from GET requests.
    public HttpResponse handleGet(HttpServletRequest request) {
        if (request == null) {
            return createResponse(400, "Bad Request", "Request was
null");
        }

        Map<String, String> formData = request.getParameters();

        // Reject the request if no parameters were supplied.
        if (formData == null || formData.isEmpty()) {
            return createResponse(400, "Bad Request", "Form data
is empty");
        }

        return handleFormData(formData);
    }

    // Shared validation and submission logic used by both GET and
POST form handling.
    private HttpResponse handleFormData(Map<String, String>
formData) {
        if (formData == null || formData.isEmpty()) {
            return createResponse(400, "Bad Request", "Form data
is empty");
        }

        // Read and safely clean the expected form fields.

```

```

        String name = getSafeValue(formData.get("name"));
        String email = getSafeValue(formData.get("email"));
        String message = getSafeValue(formData.get("message"));

        // All fields must be present and non-empty.
        if (name.isEmpty() || email.isEmpty() ||
message.isEmpty()) {
            return createResponse(400, "Bad Request", "All fields
are required");
        }

        // Create a new map containing the cleaned submission
data.
        Map<String, String> submissionData = new HashMap<>();
        submissionData.put("name", name);
        submissionData.put("email", email);
        submissionData.put("message", message);

        // Attempt to save the validated submission.
        boolean saved =
submissionStore.saveSubmission(submissionData);

        if (!saved) {
            return createResponse(500, "Internal Server Error",
"Could not save submission");
        }

        return createResponse(200, "OK", "Form submitted
successfully");
    }

    // Converts a URL-encoded form body into a map of keys and
values.
    private Map<String, String> parseFormData(String body) {
        Map<String, String> formData = new HashMap<>();

        // Split the form body into individual key-value pairs.
        String[] pairs = body.split("&");

        for (String pair : pairs) {
            if (pair == null || pair.trim().isEmpty()) {

```

```

        continue;
    }

    // Split each pair into key and value, limiting to two
parts
    // so values containing '=' are not broken
incorrectly.
    String[] keyValue = pair.split("=", 2);

    String key = decodeFormValue(keyValue[0]);
    String value = keyValue.length > 1 ?
decodeFormValue(keyValue[1]) : "";

    // Ignore empty keys and only store valid entries.
    if (!key.isEmpty()) {
        formData.put(key, value);
    }
}

return formData;
}

// Decodes URL-encoded form values such as spaces and special
characters.
private String decodeFormValue(String value) {
    if (value == null) {
        return "";
    }

    try {
        return URLDecoder.decode(value, "UTF-8");
    } catch (UnsupportedEncodingException e) {
        // UTF-8 should always be supported, so reaching this
point
        // indicates an unexpected decoding problem.
        throw new IllegalArgumentException("Invalid form
encoding");
    }
}
}

```

```

    // Returns a trimmed value, or an empty string if the input is
    null.
    private String getSafeValue(String value) {
        if (value == null) {
            return "";
        }

        return value.trim();
    }

    // Creates a plain text HTTP response with the given status
    and body.
    private HttpResponse createResponse(int statusCode, String
    reasonPhrase, String body) {
        HttpResponse response = new HttpResponse("HTTP/1.1",
    statusCode, reasonPhrase, body);
        response.addHeader("Content-Type", "text/plain;
    charset=UTF-8");
        return response;
    }
}

```

```

import java.util.Map;

public class HttpRequest {
    // Stores the HTTP method, such as GET or POST.
    private String method;

    // Stores the requested path, such as /index.html.
    private String path;

    // Stores the HTTP version from the request line.
    private String version;

    // Stores all request headers as key-value pairs.
    private Map<String, String> headers;

    // Stores the request body, mainly used for POST requests.
    private String body;

    // Stores the raw query string portion of the URL, if present.

```

```
private String queryString;

// Stores parsed query parameters as key-value pairs.
private Map<String, String> parameters;

// Creates a new HttpRequest object containing all parsed
request data.
public HttpRequest(String method, String path, String version,
                   Map<String, String> headers, String body,
                   String queryString, Map<String, String>
parameters) {
    this.method = method;
    this.path = path;
    this.version = version;
    this.headers = headers;
    this.body = body;
    this.queryString = queryString;
    this.parameters = parameters;
}

// Returns the HTTP method of the request.
public String getMethod() {
    return method;
}

// Returns the requested path.
public String getPath() {
    return path;
}

// Returns the HTTP version.
public String getVersion() {
    return version;
}

// Returns all request headers.
public Map<String, String> getHeaders() {
    return headers;
}

// Returns the request body.
```

```

    public String getBody() {
        return body;
    }

    // Returns the raw query string.
    public String getQueryString() {
        return queryString;
    }

    // Returns the parsed request parameters.
    public Map<String, String> getParameters() {
        return parameters;
    }
}

```

```

import java.nio.charset.StandardCharsets;
import java.util.LinkedHashMap;
import java.util.Map;

public class HttpResponse {

    // Stores the HTTP version for the response, such as HTTP/1.1.
    private final String httpVersion;

    // Stores the numeric status code, such as 200 or 404.
    private final int statusCode;

    // Stores the textual reason phrase, such as OK or Not Found.
    private final String reasonPhrase;

    // Stores the response headers in insertion order.
    private final Map<String, String> headers;

    // Stores the response body content.
    private final String body;

    // Creates a new HttpResponse object with the basic response
    information.
    public HttpResponse(String httpVersion, int statusCode, String
    reasonPhrase, String body) {
        this.httpVersion = httpVersion;
    }
}

```

```

        this.statusCode = statusCode;
        this.reasonPhrase = reasonPhrase;
        this.body = (body == null) ? "" : body;
        this.headers = new LinkedHashMap<>();
    }

    // Adds a header to the response.
    public void addHeader(String name, String value) {
        headers.put(name, value);
    }

    // Builds and returns the status line, for example:
    // HTTP/1.1 200 OK
    public String getStatusLine() {
        return httpVersion + " " + statusCode + " " +
reasonPhrase;
    }

    // Builds the full HTTP response as a single string,
    // including the status line, headers, blank line, and body.
    public String buildResponse() {
        // Add Content-Length automatically if it has not already
been set.
        if (!headers.containsKey("Content-Length")) {
            headers.put("Content-Length",
String.valueOf(body.getBytes(StandardCharsets.UTF_8).length));
        }

        StringBuilder response = new StringBuilder();

        // Start with the HTTP status line.
        response.append(getStatusLine()).append("\r\n");

        // Append each header on its own line.
        for (Map.Entry<String, String> header :
headers.entrySet()) {
            response.append(header.getKey())
                .append(": ")
                .append(header.getValue())
                .append("\r\n");
        }
    }

```

```

        // Add the required blank line between headers and body.
        response.append("\r\n");
        response.append(body);

        return response.toString();
    }
}

```

```

import java.io.IOException;
import java.net.ServerSocket;
import java.net.Socket;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class MainServer {

    // Starts the server, listens for incoming client connections,
    // and passes each connection to a worker thread from the
    pool.
    public void server() {
        // Fixed-size thread pool used to handle multiple client
        connections concurrently.
        ExecutorService threadPool =
        Executors.newFixedThreadPool(10);

        try (ServerSocket serverSocket = new ServerSocket(8080)) {
            AppLogger.logServerStart(8080);
            System.out.println("Server running on port 8080...");

            // Keep accepting client connections for as long as
            the server is running.
            while (true) {
                Socket socket = serverSocket.accept();

                // Create a handler for the connected client.
                ConnectionHandler connectionHandler = new
                ConnectionHandler(socket);

                // Submit the handler to the thread pool so it can
                run on a worker thread.
            }
        }
    }
}

```

```

        threadPool.submit(connectionHandler);
    }
} catch (IOException e) {
    // Log any server-level I/O failures.
    AppLogger.logInternalError("Server failed in
MainServer", e);
    e.printStackTrace();
} finally {
    // Log when the server stops.
    AppLogger.logServerStop();
}
}

// Program entry point. Creates the server object and starts
it.
public static void main(String[] args) {
    MainServer mainServer = new MainServer();
    mainServer.server();
}
}

```

```

import java.util.HashMap;
import java.util.LinkedHashMap;
import java.util.Map;

public class RequestParser {

    // Stores the raw HTTP request text received from the client.
    private final String rawRequest;

    // Creates a parser instance for the given raw request.
    public RequestParser(String rawRequest) {
        this.rawRequest = rawRequest;
    }

    // Parses the raw request string into a structured HttpRequest
object.
    public HttpRequest parse() {
        // Reject completely empty requests.
        if (rawRequest == null || rawRequest.trim().isEmpty()) {
            AppLogger.logMalformedRequest("Request was empty");

```

```
        throw new IllegalArgumentException("Request was
empty");
    }

    // Split the request into lines so the request line,
headers, and body can be processed.
    String[] lines = rawRequest.split("\\r?\\n");

    if (lines.length == 0) {
        AppLogger.logMalformedRequest("Request had no lines");
        throw new IllegalArgumentException("Request had no
lines");
    }

    // The first line should always be the HTTP request line.
    String requestLine = lines[0].trim();

    if (requestLine.isEmpty()) {
        AppLogger.logMalformedRequest("Request line was
empty");
        throw new IllegalArgumentException("Request line was
empty");
    }

    // Split the request line into method, path, and HTTP
version.
    String[] parts = requestLine.split("\\s+");

    if (parts.length != 3) {
        AppLogger.logMalformedRequest("Malformed request
line");
        throw new IllegalArgumentException("Malformed request
line");
    }

    String method = parts[0];
    String path = parts[1];
    String version = parts[2];

    // Ensure the HTTP method is present.
    if (method.trim().isEmpty()) {
```

```
        AppLogger.logMalformedRequest("HTTP method was
empty");
        throw new IllegalArgumentException("HTTP method was
empty");
    }

    // Ensure the request path is present.
    if (path.trim().isEmpty()) {
        AppLogger.logMalformedRequest("Request path was
empty");
        throw new IllegalArgumentException("Request path was
empty");
    }

    // Only HTTP versions in the expected format are accepted.
    if (!version.startsWith("HTTP/")) {
        AppLogger.logMalformedRequest("Invalid HTTP version
format");
        throw new IllegalArgumentException("Invalid HTTP
version format");
    }

    // This server is limited to HTTP/1.1 requests.
    if (!version.equals("HTTP/1.1")) {
        AppLogger.logMalformedRequest("Only HTTP/1.1 is
supported");
        throw new IllegalArgumentException("Only HTTP/1.1 is
supported");
    }

    String queryString = "";
    Map<String, String> parameters = new HashMap<>();

    // If the path contains a query string, split it away from
the main path.
    if (path.contains("?")) {
        String[] fullPath = path.split("\\?", 2);
        path = fullPath[0];
        queryString = fullPath[1];
    }
}
```

```

        // Split the query string into individual key-value
pairs.

        String[] pairs = queryString.split("&");

        for (String pair : pairs) {
            if (pair.trim().isEmpty()) {
                continue;
            }

            String[] keyValue = pair.split("=", 2);
            String key = keyValue[0].trim();
            String value = keyValue.length > 1 ?
keyValue[1].trim() : "";

            // Only store parameters with a non-empty key.
            if (!key.isEmpty()) {
                parameters.put(key, value);
            }
        }

        // Store headers in insertion order.
        Map<String, String> headers = new LinkedHashMap<>();
        int bodyStartIndex = -1;

        // Process all lines after the request line as headers
until a blank line is found.
        for (int i = 1; i < lines.length; i++) {
            String line = lines[i];

            // A blank line marks the end of the headers and the
start of the body.
            if (line.trim().isEmpty()) {
                bodyStartIndex = i + 1;
                break;
            }

            int colonIndex = line.indexOf(":");

            // Headers must contain a colon separating the name
and value.

```

```
        if (colonIndex <= 0) {
            AppLogger.logMalformedRequest("Invalid header
format");
            throw new IllegalArgumentException("Invalid header
format");
        }

        String headerName = line.substring(0,
colonIndex).trim();
        String headerValue = line.substring(colonIndex +
1).trim();

        // Reject headers with an empty name.
        if (headerName.isEmpty()) {
            AppLogger.logMalformedRequest("Header name was
empty");
            throw new IllegalArgumentException("Header name
was empty");
        }

        // Validate Content-Length if it is present.
        if (headerName.equalsIgnoreCase("Content-Length")) {
            if (headerValue.isEmpty()) {
                AppLogger.logMalformedRequest("Content-Length
header was empty");
                throw new
IllegalArgumentException("Content-Length header was empty");
            }

            try {
                int length = Integer.parseInt(headerValue);

                if (length < 0) {
                    AppLogger.logMalformedRequest("Content-Length cannot be
negative");
                    throw new
IllegalArgumentException("Content-Length cannot be negative");
                }
            } catch (NumberFormatException e) {
```

```

        AppLogger.logMalformedRequest("Invalid
Content-Length value");
        throw new IllegalArgumentException("Invalid
Content-Length value");
    }
}

headers.put(headerName, headerValue);
}

StringBuilder bodyBuilder = new StringBuilder();

// Rebuild the request body from all lines after the blank
line.
if (bodyStartIndex != -1 && bodyStartIndex < lines.length)
{
    for (int i = bodyStartIndex; i < lines.length; i++) {
        bodyBuilder.append(lines[i]);

        if (i < lines.length - 1) {
            bodyBuilder.append("\n");
        }
    }
}

String body = bodyBuilder.toString();

// Return a structured HttpRequest containing all parsed
parts of the request.
return new HttpRequest(method, path, version, headers,
body, queryString, parameters);
}
}

```

```

public class Router {

    // Handles requests for static files such as HTML pages.
    private final StaticFileService staticFileService;

    // Handles form submission routes.
    private final FormService formService;
}

```

```
// Create the router and initialise the services it delegates
to.

public Router() {
    this.staticFileService = new StaticFileService();
    this.formService = new FormService();
}

// Routes an incoming HttpRequest to the correct handler based
on method and path.
public HttpResponseMessage route(HttpRequest request) {
    // Reject null requests immediately.
    if (request == null) {
        return createErrorResponse(400, "Bad Request",
"Invalid request");
    }

    String method = request.getMethod();
    String path = request.getPath();

    // Reject requests missing essential request line
information.
    if (method == null || path == null) {
        return createErrorResponse(400, "Bad Request",
"Missing method or path");
    }

    // Log the incoming request before routing it.
    AppLogger.logRequest(method, path);

    // Handle GET requests.
    if ("GET".equals(method)) {
        // Map the root path to the default homepage.
        if ("/".equals(path)) {
            path = "/index.html";
        }

        // Send GET requests for /submit to the form service.
        if ("/submit".equals(path)) {
            return formService.handleGet(request);
        }
    }
}
```

```

        // All other GET requests are treated as static file
requests.
        return staticFileService.handleGet(path);
    }

    // Handle POST requests.
    if ("POST".equals(method)) {
        // Only the /submit route is supported for POST.
        if ("/submit".equals(path)) {
            return formService.handlePost(request);
        }

        // Any other POST path is treated as missing.
        AppLogger.logNotFound(path);
        return createErrorResponse(404, "Not Found", "POST
route not found");
    }

    // Any other HTTP method is rejected.
    AppLogger.warning("405 Method Not Allowed: " + method + "
" + path);
    return createErrorResponse(405, "Method Not Allowed",
"Method not allowed");
}

// Helper method used to create plain text error responses.
private HttpResponse createErrorResponse(int statusCode,
String reasonPhrase, String body) {
    HttpResponse response = new HttpResponse("HTTP/1.1",
statusCode, reasonPhrase, body);
    response.addHeader("Content-Type", "text/plain;
charset=UTF-8");
    return response;
}
}

```

```

import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;

```

```
public class SafePathResolver {

    // Base directory that all requested files must stay within.
    private Path base;

    // Raw path requested by the client.
    private String requestedPath;

    // Set the fixed public directory and store the requested
    path.
    public SafePathResolver(String requestedPath) {
        this.base =
Paths.get("public").toAbsolutePath().normalize();
        this.requestedPath = requestedPath;
    }

    // Resolves and validates the requested path before it is used
    for file access.
    public Path safePath() {
        // Reject null or blank paths.
        if (requestedPath == null ||
requestedPath.trim().isEmpty()) {
            return null;
        }

        String cleanedPath = requestedPath.trim();

        // Only allow paths that begin with a forward slash.
        if (!cleanedPath.startsWith("/")) {
            return null;
        }

        // Map the root request to the default homepage file.
        if (cleanedPath.equals("/")) {
            cleanedPath = "index.html";
        } else {
            // Remove the leading slash so the path can be safely
            resolved against the base directory.
            cleanedPath = cleanedPath.substring(1);
        }
    }
}
```

```

        // Resolve the requested path against the public directory
        and normalise it
        // to remove sequences such as ../
        Path resolvedPath = base.resolve(cleanedPath).normalize();

        // Reject any path that escapes outside the intended
        public directory.
        if (!resolvedPath.startsWith(base)) {
            AppLogger.logUnsafePathRejected(requestedPath);
            return null;
        }

        // Reject paths that do not point to an existing file.
        if (!Files.exists(resolvedPath)) {
            return null;
        }

        // Reject anything that is not a regular file, such as a
        directory.
        if (!Files.isRegularFile(resolvedPath)) {
            return null;
        }

        return resolvedPath;
    }
}

```

```

import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;
import java.nio.file.Path;

public class StaticFileService {

    // Default constructor for the static file service.
    public StaticFileService() {
    }

    // Handles GET requests for files stored inside the public
    directory.
    public HttpResponse handleGet(String path) {
    }
}

```

```
        // Resolve the requested path safely before attempting any
file access.

        SafePathResolver safePathResolver = new
SafePathResolver(path);
        Path safePath = safePathResolver.safePath();

        // If the resolved path is invalid, unsafe, or does not
point to a valid file,
        // return a 404 response.
        if (safePath == null) {
            AppLogger.logNotFound(path);

            HttpResponse response = new HttpResponse(
                "HTTP/1.1",
                404,
                "Not Found",
                "File not found"
            );
            response.addHeader("Content-Type", "text/plain;
charset=UTF-8");
            return response;
        }

        StringBuilder fileContent = new StringBuilder();

        // Read the requested file line by line and build the
response body.
        try (BufferedReader bufferedReader = new
BufferedReader(new FileReader(safePath.toFile()))) {
            String line;

            while ((line = bufferedReader.readLine()) != null) {
                fileContent.append(line).append("\n");
            }

            // Log successful file serving.
            AppLogger.logFileServed(safePath.toString());

            HttpResponse response = new HttpResponse(
                "HTTP/1.1",
                200,
```

```

        "OK",
        fileContent.toString()
    );
    response.addHeader("Content-Type", "text/html;
charset=UTF-8");
    return response;

    } catch (IOException e) {
        // Return a controlled 500 response if the file cannot
be read.
        AppLogger.logInternalError("Failed to read file: " +
safePath, e);

        HttpResponse response = new HttpResponse(
            "HTTP/1.1",
            500,
            "Internal Server Error",
            "Error reading file"
        );
        response.addHeader("Content-Type", "text/plain;
charset=UTF-8");
        return response;
    }
}
}

```

```

import java.io.BufferedWriter;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.nio.file.StandardOpenOption;
import java.time.LocalDateTime;
import java.util.Map;

public class SubmissionStore {

    // Shared lock used to prevent multiple threads writing to the
file at the same time.
    private static final Object FILE_LOCK = new Object();

```

```

    // Absolute path to the file where form submissions are
    stored.
    private final Path storageFile;

    // Set the storage file location when the store is created.
    public SubmissionStore() {
        this.storageFile = Paths.get("data",
"submissions.txt").toAbsolutePath().normalize();
    }

    // Saves a validated form submission to the storage file.
    public boolean saveSubmission(Map<String, String>
submissionData) {
        // Reject null or empty submission data.
        if (submissionData == null || submissionData.isEmpty()) {
            return false;
        }

        // Lock file writing so only one thread can write at a
        time.
        synchronized (FILE_LOCK) {
            try {
                Path parent = storageFile.getParent();

                // Create the parent directory if it does not
                already exist.
                if (parent != null) {
                    Files.createDirectories(parent);
                }

                // Build a single text record from the submission
                data.
                String record = buildRecord(submissionData);

                // Open the file in append mode and create it if
                it does not exist.
                try (BufferedWriter writer =
Files.newBufferedWriter(
                    storageFile,
                    StandardOpenOption.CREATE,
                    StandardOpenOption.APPEND

```

```

        )) {
            writer.write(record);
            writer.newLine();
        }

        // Log the stored submission after it has been
written successfully.
        AppLogger.logSubmissionStored(record);
        return true;

    } catch (IOException e) {
        // Log any storage failure and report that saving
did not succeed.
        AppLogger.logInternalError("Failed to store
submission", e);
        e.printStackTrace();
        return false;
    }
}

// Builds a single line record containing a timestamp and all
submitted fields.
private String buildRecord(Map<String, String> submissionData)
{
    StringBuilder record = new StringBuilder();

    record.append("timestamp=").append(LocalDateTime.now());

    for (Map.Entry<String, String> entry :
submissionData.entrySet()) {
        record.append(" | ")
            .append(entry.getKey())
            .append("=")
            .append(entry.getValue());
    }

    return record.toString();
}
}

```